

Manual del Proyecto

Integrantes

1. Manuel Politis Kufa
2. Gonzalo Alejo Dominguez
3. Ignacio Zanoni Mutti

Herramientas usadas

- IDE: Vscode
- El blog *iForce2d* compartido por la cátedra: [Box2D C++ tutorials - Introduction](#) qué fue de mucha ayuda para aprender sobre Box2D
- El blog del profesor Martín DiPaola ([The Book of Gehn](#)), en particular el artículo que explica la lógica detrás del cómo implementar un *Constant Rate Loop*
- Tilled, para todo lo relacionado a la generación de información de los mapas del juego
- Linters: Clang-format, Cpplint, Cppcheck, Pre-commit
- Valgrind, para chequeos de memoria
- Htop para medir el rendimiento de los programas desarrollados
- SDL2pp
- Qt6
- Tiburoncin ([Repositorio Github Tiburoncin](#)), especialmente durante las primeras semanas mientras íbamos definiendo el protocolo
- Google Tests

División de la tareas

Las tareas dentro del proyecto se dividieron de la siguiente manera:

Manuel:

- Desarrollo de mayor parte del cliente
- Manejo de SDL: manejo de su lógica y funciones, usar correctamente las texturas provistas por la cátedra, manejo del input de usuario y manejo de los sonidos del juego antes ciertas situaciones dentro del mismo.
- Elección, creación y edición de las imágenes usadas para el hud del juego.
- Elección de los sonidos y música del juego.
- Ayuda y ajuste en algunos detalles de la lógica del juego para que se adapte a lo que había desarrollado y necesitaba en el cliente.
- Fixes de errores de memoria generales del proyecto
- Definición de algunas parte del protocolo del proyecto

Gonzalo:

- Desarrollo de la lógica del juego en el servidor: manejo del motor de físicas Box2d y las funcionalidades que ofrece, desarrollo de la lógica del loop principal de juego, definición como se desarrolla una partida y de las entidades que forman parte de esa lógica.
- Configuración del proyecto a través de un archivo YAML.
- Creación de los datos de colisiones de los distintos mapas, junto con el diseño de los recorridos usando la herramienta antes mencionada Tiled
- Edición del CmakeFile y la compilación del proyecto
- Definición de algunas parte del protocolo del proyecto
- Fixes de errores de memoria generales del proyecto

Ignacio:

- Desarrollo de la concurrencia y el manejo de hilos: como se realiza la gestión de múltiples partidas en paralelo cada una con múltiples clientes conectados a la vez.
 - Definición de los mensajes que se envían entre cliente y servidor: Al crear una partida, cuando la misma finaliza, se envía información de input o información del servidor para que los clientes consuman.
 - Creación de todas las pantallas de QT: Su lógica y su aspecto visual, estilo, elección de fotografías de fondo, fuentes, etc.
- OBS: Esta tarea la tomó a partir de la semana 3-4 del proyecto luego de que Geronimo Fanti, nuestro compañero encargado originalmente de esta tarea, se bajara de la materia.

- Definición de parte del protocolo y como es la comunicación entre servidor y cliente/s.
- Tests del protocolo con Google Tests.

Tareas realizadas a lo largo de las semanas

Semana 1:

- División de las tareas que iba a realizar c/u
- Acercamiento a las distintas herramientas y recursos que íbamos a utilizar.

Semana 2:

- Implementación inicial del cliente de SDL
- Primera versión de la pantalla de login
- Prototipo con muy poco SDL para probar el funcionamiento del motor de físicas Box2d.
- Todavía no había conexión entre cliente y servidor.

Semana 3:

- Primera versión del protocolo de comunicación
- Logramos la primera conexión efectiva entre cliente y servidor.
- Todo lo que hace y dibuja el cliente depende de lo que le indica el servidor.
- El cliente SDL dibuja varios clientes en simultáneo que le envía el servidor
- Los autos ya tienen toda su lógica de físicas completa.
- Carga de datos de los mapas de archivo YAML

Semana 4:

- Se agrega el hud en el cliente
- Se agregan a la lógica del servicio las entidades faltantes, paredes, checkpoints además de los autos ya presentes.
- Lógica de colisiones implementada, entre autos, autos y paredes, por ultimo autos con checkpoints
- Se termina la concurrencia en el servidor



Semana 5:

- Se termina la concurrencia
- Se agrega tanto al server como al cliente tener un constant rate loop
- Se agrega en el server la lógica de ir pasando por checkpoints de manera ordenada
- Se agrega el minimapa al cliente, muestra la posición del jugador actual, la de los demás jugadores de la carrera y la posición del siguiente checkpoint.
- Se agregan hints

Semana 6:

- Agregado al lobby elección de mapa y auto
- El cliente SDL renderiza el sprite de auto de acuerdo a lo que eligió el cliente al unirse
- Se agregan sonidos al cliente SDL
- Se añade menú de compras, solo la parte visual
- Arregla de problemas de memoria pendiente
- Se calculan resultados de una carrera al terminar

Semana 7:

- Se cierra toda la lógica de compras
- El audio del juego se mejora para que tenga en cuenta distancias, además se agrega música
- Se permite más de una carrera en el mismo mapa
- Cálculo de resultados finales considerando todas las carreras
- Eliminación del mapa de los jugadores muertos o que perdieron
- Configurabilidad con YAML
- Se termina la pantalla de fin de juego con el ranking de jugadores
- Se mejora la estética en general de las pantallas de Qt

Puntos Problemáticos

- Durante las primeras semanas también tuvimos algunos problemas de comunicación y coordinación a la hora de trabajar en equipo, principalmente debido a que no nos conocíamos entre nosotros previo a la conformación de los equipos de trabajo.
- La primera gran barrera a nivel desarrollo que tuvimos que superar durante las primeras dos semanas fueron los inconvenientes que tuvimos para lograr una comunicación efectiva entre el cliente y el servidor. Lo que llevó a que inicialmente el desarrollo tanto del cliente como del servidor fuera aislado el uno del otro.
- El adaptarnos al uso de Qt, la cual era una herramienta nueva para todos lo que se encargaron de utilizarla en el desarrollo
- Lograr hacer el pase de la ventana inicial de Qt a la pantalla de SDL y viceversa al mostrar la pantalla con el ranking de jugadores.
- Poder traducir las imágenes de los mapas en información que pueda ser utilizada por el servidor.
- Atraso en la implementación de la concurrencia y las múltiples partidas

Errores conocidos

- El cliente al finalizar una carrera habiendo llegado a la meta antes de que se termine el tiempo se le muestra la pantallas con la leyenda “Has Muerto” además de que se reproduce la música correspondiente a aquellos jugadores que perdieron.
- Si el jugador presiona muchas teclas justo antes de terminar una carrera, algunas veces al pasar a la siguiente el auto del mismo avanza de forma involuntaria.

Autocrítica

Como grupo consideramos que hay ciertas cosas que podríamos haber hecho mejor a lo largo del desarrollo del trabajo y si tuviésemos que volver a hacerlo nos gustaría hacer de otra forma:

- 
- Subestimamos en cierto punto la dificultad y tiempo que nos iba a implicar el trabajo práctico, especialmente durante las primeras semanas donde nuestro nivel de avance semana a semana era un tanto bajo. Por suerte pudimos corregir y revertir en las últimas semanas la situación, donde avanzamos considerablemente en un periodo ajustado de tiempo. Pero si es cierto que haber arrancado de mejor manera nos hubiese sacado mucho estrés de esta última etapa.
 - Haber tenido una no muy buena comunicación durante las primeras semanas. Como ya mencionamos no nos conocíamos antes de que conformamos el equipo de trabajo, por lo que al principio la comunicación fue un tanto escueta y nos costaba encontrar una dinámica de trabajo clara. En especial nos costó el poder dividir bien los tiempos y coordinar que iba a estar haciendo cada uno para luego mergearlo a la rama main de nuestro repositorio.
 - No haber sido tan rigurosos con la prolijidad del código presentado. Al tener que avanzar con rapidez quedaron en segundo plano detalles como estar ejecutando y chequeando continuamente los linters y pensando en cómo modularizar mejor el código.

■ ■ ■